

INFORMACION ADICIONAL

► Constantes y más

• Límites de tipos

|            |                                                                            |
|------------|----------------------------------------------------------------------------|
| INT_MAX    | 2,147,483,647 = 2 <sup>31</sup> − 1 ≈ 2.1 × 10 <sup>9</sup>                |
| INT_MIN    | −2 <sup>31</sup> = −2,147,483,648                                          |
| UINT_MAX   | 4,294,967,295 = 2 <sup>32</sup> − 1 ≈ 4.29 × 10 <sup>9</sup>               |
| LLONG_MAX  | 9,223,372,036,854,775,807 = 2 <sup>63</sup> − 1 ≈ 9.22 × 10 <sup>18</sup>  |
| LLONG_MIN  | −2 <sup>63</sup>                                                           |
| ULLONG_MAX | 2 <sup>64</sup> − 1 = 18,446,744,073,709,551,615 ≈ 1.84 × 10 <sup>19</sup> |
| double     | enteros exactos hasta 2 <sup>53</sup> ≈ 9.007 × 10 <sup>15</sup>           |

• Valores de INF recomendados

int: INF = 10<sup>9</sup> (o 0x3f3f3f3f ≈ 1.06 × 10<sup>9</sup>, permite INF + INF sin overflow).

long long: INF = 10<sup>18</sup> (o 4e18; ojo con sumar dos INF).

• Potencias de 2

|                 |                             |                 |                           |
|-----------------|-----------------------------|-----------------|---------------------------|
| 2 <sup>10</sup> | 1,024                       | 2 <sup>30</sup> | 1,073,741,824             |
| 2 <sup>16</sup> | 65,536                      | 2 <sup>31</sup> | 2,147,483,648             |
| 2 <sup>20</sup> | 1,048,576 ≈ 10 <sup>6</sup> | 2 <sup>32</sup> | 4,294,967,296             |
| 2 <sup>24</sup> | 16,777,216                  | 2 <sup>40</sup> | ≈ 1.1 × 10 <sup>12</sup>  |
| 2 <sup>25</sup> | 33,554,432                  | 2 <sup>50</sup> | ≈ 1.13 × 10 <sup>15</sup> |
| 2 <sup>27</sup> | 134,217,728                 | 2 <sup>60</sup> | ≈ 1.15 × 10 <sup>18</sup> |

• Memoria

1 KB = 1024 bytes, 1 MB = 1024 KB = 1024<sup>2</sup> bytes = 2<sup>20</sup> bytes.  
int = 4 B, long long = 8 B, double = 8 B, char/bool = 1 B.  
10<sup>6</sup> ints ≈ 4 MB; 10<sup>7</sup> ints ≈ 40 MB; con 256 MB caben ≈ 6 × 10<sup>7</sup> ints.

• ASCII

’0’=48, ’A’=65, ’a’=97 (’a’−’A’=32, es decir, el bit 5).

• Complejidad según *n* (límite ≈ 10<sup>8</sup> ops simples/s)

|                                                 |                                              |
|-------------------------------------------------|----------------------------------------------|
| <i>n</i> ≤ 10                                   | <i>O</i> ( <i>n</i> !)                       |
| <i>n</i> ≤ 20–25                                | <i>O</i> (2 <sup><i>n</i></sup> · <i>n</i> ) |
| <i>n</i> ≤ 100                                  | <i>O</i> ( <i>n</i> <sup>4</sup> )           |
| <i>n</i> ≤ 500                                  | <i>O</i> ( <i>n</i> <sup>3</sup> )           |
| <i>n</i> ≤ 5,000                                | <i>O</i> ( <i>n</i> <sup>2</sup> )           |
| <i>n</i> ≤ 10 <sup>5</sup> –2 × 10 <sup>5</sup> | <i>O</i> ( <i>n</i> log <i>n</i> )           |
| <i>n</i> ≤ 10 <sup>6</sup> –10 <sup>7</sup>     | <i>O</i> ( <i>n</i> )                        |
| <i>n</i> ≤ 10 <sup>18</sup>                     | <i>O</i> (log <i>n</i> ) o <i>O</i> (1)      |

• Factoriales y Fibonacci (máximo que cabe)

12! = 479,001,600 es el mayor en int; 13! > INT\_MAX.

20! = 2,432,902,008,176,640,000 es el mayor en long long.

*F*<sub>46</sub> = 1,836,311,903 mayor en int;

*F*<sub>92</sub> = 7,540,113,804,746,346,429 mayor en long long.

10! = 3,628,800,

Catalan: *C*<sub>10</sub> = 16,796, *C*<sub>15</sub> = 9,694,845, *C*<sub>19</sub> = 1,767,263,190 (mayor en int).

• Máximo número de divisores

*n* ≤ 10<sup>6</sup>: 240 (720720) *n* ≤ 10<sup>9</sup>: 1344 (735134400) *n* ≤ 10<sup>18</sup>: 103,680 (897612484786617600).

• Primos y módulos comunes

10<sup>9</sup> + 7, 998,244,353 = 119 · 2<sup>23</sup> + 1 (NTT), 10<sup>9</sup> + 9.

• Cantidades de primos ≤ *n*:

| <i>n</i>        | π( <i>n</i> ) | <i>n</i>         | π( <i>n</i> )      |
|-----------------|---------------|------------------|--------------------|
| 10 <sup>2</sup> | 25            | 10 <sup>9</sup>  | 50,847,534         |
| 10 <sup>3</sup> | 168           | 10 <sup>10</sup> | 455,052,511        |
| 10 <sup>4</sup> | 1,229         | 10 <sup>11</sup> | 4,118,054,813      |
| 10 <sup>5</sup> | 9,592         | 10 <sup>12</sup> | 37,607,912,018     |
| 10 <sup>6</sup> | 78,498        | 10 <sup>13</sup> | 346,065,536,839    |
| 10 <sup>7</sup> | 664,579       | 10 <sup>14</sup> | 3,204,941,750,802  |
| 10 <sup>8</sup> | 5,761,455     | 10 <sup>15</sup> | 29,844,570,422,669 |

• Constantes reales

π ≈ 3.14159265358979, *e* ≈ 2.71828182845905, √2 ≈ 1.41421356237,

φ =  $\frac{1+\sqrt{5}}{2}$  ≈ 1.6180339887, ln 2 ≈ 0.693147, γ ≈ 0.5772156649.  
acos(−1) da π con precisión de double.

► Equivalencias

En general, para enteros positivos int a,b,x (o long long):

$$\text{if}( \text{a}*\text{b} > \text{x} ) \equiv \text{if}( \text{a} > \text{x} / \text{b} )$$

Con >= debería ser

$$\text{if}( \text{a} >= (\text{x} + \text{b} - 1) / \text{b} ) \equiv \text{if}( \text{a} > (\text{x} - 1) / \text{b} ) \\ \equiv \text{if}( \text{a}*\text{b} >= \text{x} )$$

• Aritmética entera

- Techo (positivos): con *a* ≥ 1  
 $[a/b] = (a + b - 1) / b = (a - 1) / b + 1$
- Módulo no negativo: ((*a*% *m*) + *m*)% *m*. En C++ −7 / 2 = −3 (trunca hacia 0) y −7% 2 = −1.
- Resta modular: (*a* − *b* + *m*)% *m*.
- Inverso modular (*m* primo): *a*<sup>−1</sup> ≡ *a*<sup>*m*−2</sup> (mód *m*).
- lcm(*a*,*b*) = *a* / gcd(*a*,*b*) \* *b* (dividir primero evita overflow).
- Overflow de suma: *a* + *b* > MAX ≡ *a* > MAX − *b*.
- Overflow de producto: usar la equivalencia de arriba, o \_\_int128.
- (*a* + *b*) / 2 puede desbordar; usar *a* + (*b* − *a*) / 2.

IDEAS <- <- <-

► DP

Elegir el estado No siempre pienses en prefijos [0,*i*]; considera también sufijos.

- dp*[*i*][...] = es posible sobrevivir las primeras *i* horas, [1,*i*].
- dp*[*i*][...] = es posible sobrevivir desde el fin de la hora *i* hasta el final, (*i*, *n* + 1].

Transiciones sobre el último elemento Revisar si ...

- se usa el último elemento o no.
- se usa el último elemento un número par o impar de veces.

Distinguir entre dos formulaciones *dp*[...][*i*] :

- dp*[...][*i*] = ya completé los primeros *i* elementos.
- dp*[...][*i*] = el *i*-ésimo elemento queda completado/terminado.

Probar quien es el ultimo elemento Si *n* ≤ 20 y el orden importa, la transición desde *dp*[*mask*] decide qué elemento se agrega al final.

► Búsqueda binaria

**Búsqueda binaria sobre la respuesta** Lograr *S* en a lo sumo *K* pasos: binaria sobre *S*. Para *S* fijo, calcular *f*(*S*), la cantidad mínima de pasos para obtener *S*, y buscar máx{*S* : *f*(*S*) ≤ *K*}. **Requisito:** *f* debe ser monótona (si se puede lograr *S*, se puede lograr cualquier *S*' < *S*).

► Ideas más generales

Ciclos Avanzar un paso a la vez, no varios.

Local Pasar de condiciones locales a condiciones globales.

**Independencia** (no probabilística): decisiones o eventos que no se afectan entre sí, por ejemplo poner G y T.

Bases o generadores:

- una base *B* de un espacio vectorial;
- los generadores de una σ-álgebra, σ(*C*).

**Informacion** En el estado actual, ¿qué información se puede olvidar y cuál es necesario conservar para resolver el problema?

► Ideas específicas

No buscar todo Los problemas “South America 2025 - Brazil First Phase **K** - Knockout” y “(NWERC 2025) **D** Dreamcatcher” mostraron que no hace falta probar toda la fuerza bruta si no algunos “lugares cercanos a donde parece que debería estar el maximo”. En lugar de probar *O*(*N*) probar una **cierta cantidad**, o empezar a probar desde “el lugar donde este el maximo”

COSAS TRIVIALES A TOMAR EN CUENTA

► ¿Qué hacer si da **WA (Wrong Answer)**?

- Revisar que se esta imprimiendo. Otros couts, formato.
- Quitar ifstream cin(in.txt); y revisar DEBUG
- Revisar Overflow:** revisar los tipos de datos (int, int64\_t).
- Revisar línea a línea:** asegúrate de que cada linea si este correcta, por ejemplo fact[*n*] y no solo n.
- Casos extremos:** prueba con casos límites, como *N* = 1, o valores muy grandes.
- Los fors estan bien: no se sale o empieza antes. Por ejemplo en divisibilidad y priomos tener cuidado si se inicia en *i*=1 en lugar de *i*=2
- Revisar los limites del problema y los limites que se pusieron en el codigo tipo MAXN= , r= , etc.
- Revisar como llenas la dp, o como llenas arreglos, sobretodo si es inplace, no es lo mismo 0 → *n* que *n* → 0.
- ...

► ¿Qué hacer si da **RT (Runtime error)**?

- Declarar arreglos fuera del main.
- Si usas modulos, ojo con las restas aunque creas que da postivio. *A* > *B* **NO** implica (*A* mód *m*) > (*B* mód *m*).
- ...

► ¿Qué hacer si da **TLE (Time Limit Excede)**?

- Quitar el Print Debugger
- Revisar que si este usando

```
1 ios_base::sync_with_stdio(0);
2 cin.tie(0);
```

```
3 cout.tie(0);

■ Ver que nada se esta ciclando
■

► ¿Qué hacer si da MLE (Memory Limit Exceeded) ?
1 #include <windows.h>
2 #include <psapi.h>
3 void check_memory() {
4     PROCESS_MEMORY_COUNTERS_EX pmc;
5     GetProcessMemoryInfo(GetCurrentProcess(),
6         (PROCESS_MEMORY_COUNTERS*)&pmc, sizeof(pmc));
7     SIZE_T physMemUsedByMe = pmc.WorkingSetSize;
8     std::cout << "memoria: " << physMemUsedByMe / 1024
9         << " KB (" << physMemUsedByMe / (1024 * 1024) << " MB)"
10         << std::endl;
11 }
```

Y para Linux /Ubuntu (no provado aun)

```
1 #include <sys/resource.h>
2 #include <iostream>
3
4 void check_memory(){
5     struct rusage usage;
6     getrusage(RUSAGE_SELF, &usage);
7     std::cout << "memoria: " << usage.ru_maxrss << " KB ("
8         << usage.ru_maxrss / 1024 << " MB)" <<
9         << std::endl;
10 }
```

tambien se puede desde bash. Buscar *Maximum resident set size (kbytes)*.

```
1 g++ -O3 j.cpp -o j
2 /usr/bin/time -v ./j < jenga.in
```

► Memoria

T arr[N]  $N \times \text{sizeof}(T)$  – La más eficiente (contigua).

vector<T>  $N \times \text{sizeof}(T)$  – +32 bytes de overhead del objeto.

vector<bool>  $\approx N/8$  bytes – Optimizado (1 bit por elemento).

bitset<N>  $\approx N/8$  bytes – Estático, el más ligero de todos.

set<T>  $\approx 48N$  bytes – Árbol: 3 punteros + color + dato + padding.

map<K, V>  $\approx 48N$  bytes – Similar al set, guarda pares std::pair.

unordered\_set<T>  $\approx 32N$  bytes – Tabla Hash: puntero por nodo + overhead de buckets.

list<T>  $\approx 24N + \text{sizeof}(T) \times N$  – Lista doblemente enlazada (2 punteros).

deque<T>  $\approx N \times \text{sizeof}(T)$  – Fragmentada en bloques, más pesada que vector.

priority\_queue<T>  $N \times \text{sizeof}(T)$  – Usa un vector por debajo (Heap).

set<T> 32 a 48 bytes+sizeof(T) (Memoria por nodo, aprox. 64-bit).

map<K, V> 32 a 48 bytes+sizeof(K)+sizeof(V) (Memoria por nodo, aprox. 64-bit).

unordered\_set<T> 24 a 32 bytes + sizeof(T) (Memoria por nodo, aprox. 64-bit).

PLANTILLA PYTHON

```
1 # librerias
2 import os, random, re, sys
3 from collections import deque
4 # INPUT & FAST
5 # input
6 sys.stdin = open("in.txt", 'r', encoding='utf-8') #<---
7     <-- comentarxd
8 # fast leer
9 input = sys.stdin.readline
10 # PARA DEBUG
11 DEB = False
12 DEB = True # comentar
13 def debug(*args, **kwargs):
14     if DEB:
15         print(*args, **kwargs)
16 # Leer y resolver
17 def solve():
18     n,m = map(int, input().split()) # separa
19     n = int( input().rstrip() ) #quita "\n"
20     h = list( map(int, input().rstrip().split() ) )
21
22     # escribir codigo aqui
23
24 # MAIN
25 if __name__ == "__main__":
26     t = int( input() )
27     for i in range(t):
28         solve()
```

► Fenwick Tree

Query puntual, actualizaciones en rangos.  $O(\log n)$

```
1 class Fenwick:
2     def __init__( self, n ):
3         self.bit = [0] * (n+1)
4         self.n = n
5     def add( self, i, val ): # agrega puntual
6         i += 1
7         while i <= self.n:
8             self.bit[i] += val
9             i += i & (-i)
10    def add_rango( self, l, r, val ): # en rango
11        self.add( l, val )
12        if r+1 < self.n :
13            self.add( r+1, -val )
14    def query_puntual( self, i ): # query para un solo punto
15        res = 0
16        i += 1
17        while i > 0:
18            res += self.bit[i]
19            i -= i & (-i)
20        return res
21
22
23 def solve():
24     n,m = map(int, input().split())
25     arbol = Fenwick(n)
26     for _ in range(m):
27         a,b, val = map(int, input().split())
28         arbol.add_rango( l=a-1, r=b-1, val = val)
29
30     for i in range(n):
31         print( arbol.query_puntual(i) )
```

MODELADO

► Distribuciones

**Discretas**  
**Bernoulli**  $X \sim \text{Ber}(p)$ ,  $p \in (0, 1)$ ;  $\mathcal{S}_X = \{0, 1\}$ ;  $f(x) = p^x(1-p)^{1-x}$ ;  $\mathbb{E} = p$ ;  $\text{Var} = p(1-p)$ ;  $M(t) = 1 - p + pe^t$ .

**Binomial**  $X \sim \text{Bin}(n, p)$ ;  $\mathcal{S}_X = \{0, \dots, n\}$ ;  $f(x) = \binom{n}{x} p^x(1-p)^{n-x}$ ;  $\mathbb{E} = np$ ;  $\text{Var} = np(1-p)$ ;  $M(t) = (1 - p + pe^t)^n$ .

**Geométrica** (ensayos hasta el 1.er éxito)  $X \sim \text{Geo}(p)$ ,  $p \in (0, 1]$ ;  $\mathcal{S}_X = \{1, 2, \dots\}$ ;  $f(x) = (1-p)^{x-1}p$ ;  $\mathbb{E} = \frac{1}{p}$ ;  $\text{Var} = \frac{1-p}{p^2}$ ;  $M(t) = \frac{pe^t}{1-(1-p)e^t}$ ,  $t < -\ln(1-p)$ .

**Bin. Negativa** (ensayos hasta el  $r$ -ésimo éxito)  $X \sim \text{NB}(r, p)$ ;  $\mathcal{S}_X = \{r, r+1, \dots\}$ ;  $f(x) = \binom{x-1}{r-1} p^r(1-p)^{x-r}$ ;  $\mathbb{E} = \frac{r}{p}$ ;  $\text{Var} = \frac{r(1-p)}{p^2}$ ;  $M(t) = \left(\frac{pe^t}{1-(1-p)e^t}\right)^r$ ,  $t < -\ln(1-p)$ .

**Poisson**  $X \sim \text{Poi}(\lambda)$ ,  $\lambda > 0$ ;  $\mathcal{S}_X = \{0, 1, \dots\}$ ;  $f(x) = \frac{e^{-\lambda} \lambda^x}{x!}$ ;  $\mathbb{E} = \text{Var} = \lambda$ ;  $M(t) = e^{\lambda(e^t-1)}$ .

**Hipergeom.**  $X \sim \text{Hyp}(N, K, n)$ ;  $\mathcal{S}_X = \{\max(0, n-N+K), \dots, \min(n, K)\}$ ;  $f(x) = \frac{\binom{K}{x} \binom{N-K}{n-x}}{\binom{N}{n}}$ ;  $\mathbb{E} = \frac{nK}{N}$ ;  $\text{Var} = n \frac{K}{N} (1 - \frac{K}{N}) \frac{N-n}{N-1}$ ;  $M(t)$  sin forma elemental ( ${}_2F_1$ ).  
**Continuas**

**Uniforme**  $X \sim \mathcal{U}(a, b)$ ;  $\mathcal{S}_X = [a, b]$ ;  $f(x) = \frac{1}{b-a}$ ;  $\mathbb{E} = \frac{a+b}{2}$ ;  $\text{Var} = \frac{(b-a)^2}{12}$ ;  $M(t) = \frac{e^{tb}-e^{ta}}{t(b-a)}$  ( $t \neq 0$ ),  $M(0) = 1$ .

**Exponencial**  $X \sim \text{Exp}(\lambda)$ ,  $\lambda > 0$ ;  $\mathcal{S}_X = [0, \infty)$ ;  $f(x) = \lambda e^{-\lambda x}$ ;  $\mathbb{E} = \frac{1}{\lambda}$ ;  $\text{Var} = \frac{1}{\lambda^2}$ ;  $M(t) = \frac{\lambda}{\lambda-t}$ ,  $t < \lambda$ .

**Normal**  $X \sim \mathcal{N}(\mu, \sigma^2)$ ,  $\sigma > 0$ ;  $\mathcal{S}_X = \mathbb{R}$ ;  $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2}$ ;  $\mathbb{E} = \mu$ ;  $\text{Var} = \sigma^2$ ;  $M(t) = e^{\mu t + \sigma^2 t^2/2}$ .

**Gamma** (forma-tasa)  $X \sim \text{Gamma}(\alpha, \beta)$ ;  $\mathcal{S}_X = (0, \infty)$ ;  $f(x) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}$ ;  $\mathbb{E} = \frac{\alpha}{\beta}$ ;  $\text{Var} = \frac{\alpha}{\beta^2}$ ;  $M(t) = (\frac{\beta}{\beta-t})^\alpha$ ,  $t < \beta$ .

**Beta**  $X \sim \text{Beta}(\alpha, \beta)$ ;  $\mathcal{S}_X = [0, 1]$ ;  $f(x) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha, \beta)}$ ,  $B = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$ ;  $\mathbb{E} = \frac{\alpha}{\alpha+\beta}$ ;  $\text{Var} = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$ ;  $M(t) = {}_1F_1(\alpha; \alpha+\beta; t)$ .

► Casos de uso y modelación

**Guía rápida: señal**  $\rightarrow$  **modelo** “ $n$  intentos iguales e independientes, contar éxitos”  $\rightarrow$  Bin; “intentos hasta el primer/ $r$ -ésimo éxito”  $\rightarrow$  Geo/NB; “eventos raros en un intervalo”, sin  $n$  explícito  $\rightarrow$  Poi; “sacar sin reemplazo”  $\rightarrow$  Hyp; “punto/instante al azar”  $\rightarrow$   $\mathcal{U}$ ; “tiempo hasta que ocurre” con tasa constante  $\rightarrow$  Exp; “tiempo hasta el  $k$ -ésimo evento”  $\rightarrow$  Gamma; “suma de muchas partes pequeñas”  $\rightarrow$   $\mathcal{N}$  (aproximación); “ $p$  desconocida” o “ $p$  uniforme”  $\rightarrow$  Beta (Laplace); “solo importa el orden relativo”  $\rightarrow$  permutación uniforme (intercambiabilidad). **Antes de modelar:** (i) ¿hay independencia o solo intercambiabilidad? (ii) ¿la linealidad de  $\mathbb{E}$  resuelve sin conocer la distribución? (iii) ¿se pide exactitud o tolerancia  $10^{-6}$ ?

**Ber/Bin Señal:**  $n$  intentos independientes con la misma  $p$ ; indicador de un evento. **Modelo:**  $X = \sum_{j \leq n} X_j$ ,  $X_j \stackrel{iid}{\sim} \text{Ber}(p)$ ;  $\mathbb{P}(X \geq 1) = 1 - (1-p)^n$  (complemento). **Relaciones:**  $\text{Bin}(n, p) + \text{Bin}(m, p) = \text{Bin}(n+m, p)$  (indep.);  $\approx \text{Poi}(np)$  si  $n \rightarrow \infty$ ,  $np \rightarrow \lambda$ ;  $\approx \mathcal{N}(np, np(1-p))$  si  $np(1-p) \gtrsim 10$  (con corrección de continuidad  $\pm \frac{1}{2}$ ); si  $X \sim \text{Bin}(n, p)$ ,  $Y \sim \text{Bin}(m, p)$  indep., entonces  $X \mid (X+Y=s) \sim \text{Hyp}$ . **En CP:**  $\mathbb{E}[\sum X_j] = \sum \mathbb{E}[X_j]$  vale aunque haya dependencia (la varianza sí exige independencia); pmf con recurrencia  $f(k+1) = f(k) \frac{n-k}{k+1} \frac{p}{1-p}$  en  $O(n)$ , o factoriales precalculados (mod  $M$ ) en  $O(1)$  por término;  $n$  grande en reales  $\Rightarrow$  logaritmos de factoriales (**lgamma**).

**Geo Señal:** “cuántos intentos hasta el primer éxito”, “se repite hasta que sale”, reinicio tras fallo. **Modelo:** tiempo de espera discreto;  $\mathbb{P}(X > k) = (1-p)^k$ ; es la única discreta sin memoria:  $\mathbb{P}(X > m+k \mid X > m) = \mathbb{P}(X > k)$ . **Relaciones:**  $\min_i \text{Geo}(p_i) \sim \text{Geo}(1 - \prod_i (1-p_i))$  (indep.);  $\text{Geo}(\lambda/n)/n \rightarrow \text{Exp}(\lambda)$ . **En CP:** en un DP, el autolazo se despeja como  $E = c + (1-p)E + \dots \Rightarrow E = \frac{c+\dots}{p}$  (no hay que iterar); muestreo por inversión  $\lceil \ln U / \ln(1-p) \rceil$ .

**NB Señal:** “intentos hasta acumular  $r$  éxitos”. **Modelo:**  $X = \sum_{j \leq r} G_j$ ,  $G_j \stackrel{iid}{\sim} \text{Geo}(p)$ ; en la pmf el último ensayo es éxito y los  $r-1$  anteriores se reparten entre las  $x-1$  posiciones. **Relaciones:** dualidad  $\mathbb{P}(\text{NB}(r, p) \leq x) = \mathbb{P}(\text{Bin}(x, p) \geq r)$ ; versión por fracasos  $= X - r$  (soporte desde 0; cuidar la convención). **En CP:** la dualidad evita un DP sobre “éxitos acumulados” y da la cola en  $O(r)$  sumando una binomial.

**Poisson Señal:** conteo de eventos raros e independientes en un intervalo, dado el promedio  $\lambda$  pero no  $n$ ; llegadas a una cola. **Modelo:**  $N(t) \sim \text{Poi}(\lambda t)$  con conteos independientes en intervalos disjuntos. **Relaciones:** superposición:  $\text{Poi}(\lambda_1) + \text{Poi}(\lambda_2) \sim \text{Poi}(\lambda_1 + \lambda_2)$ ; *thinning*: conservar cada evento con prob.  $q$  da  $\text{Poi}(q\lambda)$ , y conservados y descartados son independientes;  $N_1 \mid (N_1+N_2=s) \sim \text{Bin}(s, \frac{\lambda_1}{\lambda_1+\lambda_2})$ ;  $\text{Bin}(n, p) \approx \text{Poi}(np)$ . **En CP:** recurrencia  $f(k+1) = f(k) \frac{\lambda}{k+1}$  y truncar la serie hacia  $k \approx \lambda + c\sqrt{\lambda}$ ; *Poissonización*: con  $n$  fijo los conteos por categoría son dependientes, pero con  $N \sim \text{Poi}(\lambda)$  son independientes  $\Rightarrow$  se calcula fácil y se des-poissoniza.

**Hyp Señal:** elegir  $n$  de  $N$  elementos distinguibles sin reemplazo (cartas, lotes, equipos). **Modelo:** cada elemento queda elegido con prob.  $\frac{n}{N}$  por simetría, así que  $\mathbb{E}[\#\text{rojos elegidos}] = \frac{nK}{N}$  sin importar la dependencia; la  $n$ -ésima extracción es roja con prob.  $\frac{K}{N}$  (intercambiabilidad), pero condicionada a  $g$  rojas previas es  $\frac{K-g}{N-(n-1)}$ . **Relaciones:** si  $N \rightarrow \infty$  y  $K/N \rightarrow p$ ,  $\rightarrow \text{Bin}(n, p)$ ; versión multivariada para varias categorías. **En CP:**  $\mathbb{P} = \binom{K}{k} \binom{N-K}{n-k} / \binom{N}{n}$  con factoriales modulares en  $O(1)$ ; para probabilidades de configuraciones, contar casos favorables / totales evita DP.

**Uniforme Señal:** “punto/instante al azar”, sin preferencia; “permutación aleatoria”. **Modelo:** probabilidad  $\propto$  longitud, área o volumen (problemas geométricos se reducen a áreas en el cuadrado unidad); si solo importa el orden de  $n$  uniformes iid continuas, todas

las  $n!$  permutaciones son equiprobables. **Relaciones:**  $U \sim \mathcal{U}(0, 1) \Rightarrow F^{-1}(U) \sim F$  (transformada inversa);  $-\ln U \sim \text{Exp}(1)$ ; estadísticos de orden  $\rightarrow$  Beta. **En CP:** “probabilidad de que el elemento  $j$  sea el mayor/récord” se resuelve por simetría sin integrar; para áreas complicadas, integral en  $O(1)$  o partición en regiones.

**Exp Señal:** “tiempo hasta que ocurre” con tasa constante; varios procesos compitiendo en paralelo; “falta de memoria”. **Modelo:** cadena de Markov en tiempo continuo: en el estado  $s$  se espera  $\text{Exp}(\Lambda_s)$ ,  $\Lambda_s = \sum \lambda_{s \rightarrow s'}$ , y se salta a  $s'$  con prob.  $\frac{\lambda_{s \rightarrow s'}}{\Lambda_s}$ . **Relaciones:**  $\min_i T_i \sim \text{Exp}(\sum \lambda_i)$ , y  $\mathbb{P}(T_k = \min) = \frac{\lambda_k}{\sum \lambda_i}$  (indep.); el

máximo de  $n$  iid  $\text{Exp}(\lambda)$  tiene  $\mathbb{E} = \frac{H_n}{\lambda}$ ; los tiempos entre llegadas de un proceso de Poisson son  $\text{Exp}$  (dualidad). **En CP:** se convierte en DP discreto sobre la cadena embebida:  $E[s] = \frac{1}{\Lambda_s} + \sum_{s'} \frac{\lambda_{s \rightarrow s'}}{\Lambda_s} E[s']$ , sin integrales ni reales continuos.

**Normal Señal:** suma de muchas contribuciones pequeñas e independientes (TLC); enunciado que tolera error  $\sim 10^{-6}$  con  $n$  enorme. **Modelo:**  $\sum X_j \approx \mathcal{N}(n\mu, n\sigma^2)$ ; estandarizar con  $Z = \frac{X-\mu}{\sigma}$  y usar  $\Phi$ . **Relaciones:** cerrada bajo suma independiente:  $\mathcal{N}(\mu_1, \sigma_1^2) + \mathcal{N}(\mu_2, \sigma_2^2) = \mathcal{N}(\mu_1+\mu_2, \sigma_1^2+\sigma_2^2)$ ; aproxima Bin y Poi; Berry-Esseen: error  $O(1/\sqrt{n})$ , malo en colas. **En CP:** es una *aproximación*; si piden exactitud o respuesta modular no se usa.  $\Phi(x) = \frac{1}{2} \text{erfc}(-x/\sqrt{2})$  en C++; para estimar tamaños o cotas es suficiente.

**Gamma Señal:** “tiempo hasta el  $k$ -ésimo evento” en un proceso de Poisson. **Modelo:** Erlang:  $T = \sum_{j \leq k} \text{Exp}(\lambda)$ . **Relaciones:** dualidad  $\mathbb{P}(T \leq t) = \mathbb{P}(\text{Poi}(\lambda t) \geq k)$  (la integral se vuelve una suma finita de  $k$  términos);  $\text{Gamma}(\alpha, \beta) + \text{Gamma}(\alpha', \beta) = \text{Gamma}(\alpha+\alpha', \beta)$  (indep., misma tasa); si  $X \sim \text{Gamma}(\alpha)$ ,  $Y \sim \text{Gamma}(\beta)$  indep.,  $\frac{X}{X+Y} \sim \text{Beta}(\alpha, \beta)$  e independiente de  $X+Y$ . **En CP:** cola exacta en  $O(k)$ ; normalizar  $n$  exponenciales iid da un punto uniforme en el simplejo ( $\text{Dirichlet}(1, \dots, 1)$ ).

**Beta Señal:** probabilidad desconocida  $p$  (“uniforme”  $\Rightarrow \text{Beta}(1,1)$ ); posición del  $k$ -ésimo punto de  $n$  uniformes. **Modelo:** prior conjugado de Ber/Bin con  $a$  éxitos y  $b$  fracasos virtuales:  $p \mid (g \text{ de } i) \sim \text{Beta}(a+g, b+i-g)$  y  $\mathbb{P}(X_{i+1}=1 \mid g \text{ de } i) = \frac{a+g}{a+b+i}$  (Laplace si  $a=b=1$ :  $\frac{g+1}{i+2}$ ). **Relaciones:**  $U_{(k)} \sim \text{Beta}(k, n-k+1)$ ; generalización a  $k$  categorías: Dirichlet; urna de Pólya  $\equiv \text{Beta-Binomial}$ . **En CP:** DP sobre  $(i, g)$  con transición  $\frac{a+g}{a+b+i}$  (inversos modulares de  $a+b+i$  en  $O(N^2)$ ; si solo importa el total de éxitos con  $\text{Beta}(1,1)$ , es uniforme en  $\{0, \dots, n\}$  y no hace falta DP.

**Mapa de relaciones**  $\text{Ber} \xrightarrow{\Sigma} \text{Bin} \rightarrow \text{Poi}$  (raro) /  $\mathcal{N}$  (TLC);  $\text{Geo} \xrightarrow{\Sigma} \text{NB}$ ;  $\text{Exp} \xrightarrow{\Sigma} \text{Gamma}$ ;  $\text{Geo} \xrightarrow{n \rightarrow \infty} \text{Exp}$ ;  $\text{Poi} \leftrightarrow \text{Exp}$  (conteo  $\leftrightarrow$  espera);  $\text{Bin} \leftrightarrow \text{NB}$ ,  $\text{Gamma} \leftrightarrow \text{Poi}$  (dualidades de colas);  $\text{Hyp} \xrightarrow{N \rightarrow \infty} \text{Bin}$ ;  $\text{Gamma}/\text{Gamma} \rightarrow \text{Beta}$ ;  $\text{Beta} \leftrightarrow \text{Bin}$  (conjugación) y  $U_{(k)} \sim \text{Beta}$ ;  $\text{Exp}(1) = -\ln \mathcal{U}$ .

► Modelacion avanzada

**Laplace / Beta-Binomial (1) Contexto.** Un evento (servidor falla, aparece un monstruo, moneda cae cara) ocurre con probabilidad fija pero desconocida  $p$ ; tras  $i$  intentos con  $g$  éxitos se

pide la probabilidad del siguiente, o un DP sobre estados  $(i, g)$ .

**(2) Modelo.**  $\theta \sim \text{Beta}(a, b)$ ,  $X_j \mid \theta \stackrel{\text{iid}}{\sim} \text{Ber}(\theta)$ ,  $S_i = \sum_{j \leq i} X_j$  (los  $X_j$  son intercambiables, no independientes). **(3) Resultado.**  $\theta \mid S_i = g \sim \text{Beta}(a + g, b + i - g)$ ;  $\mathbb{P}(X_{i+1} = 1 \mid S_i = g) = \frac{a+g}{a+b+i}$ ;  $\mathbb{P}(S_n = k) = \binom{n}{k} \frac{B(a+k, b+n-k)}{B(a, b)}$ ; una secuencia concreta con  $k$  éxitos tiene probabilidad  $\frac{B(a+k, b+n-k)}{B(a, b)}$ . Con  $a=b=1$ :  $\frac{g+1}{i+2}$  y  $S_n$  uniforme en  $\{0, \dots, n\}$ . **(4) Idea.** Integral beta  $\int_0^1 \theta^x (1-\theta)^y d\theta = \frac{x! y!}{(x+y+1)!}$  (por partes) y Bayes: posterior  $\propto$  verosimilitud  $\times$  prior; Beta es conjugada de Bernoulli. Alternativa sin integrales: argumento de los huecos con  $i+2$  uniformes. **(5) CP.**  $O(1)$  por transición, DP completo  $O(N^2)$  estados  $(i, g)$ ; usar inversos modulares de  $a+b+i$ . Como la probabilidad de una secuencia solo depende de  $k$ , se puede colapsar el historial a  $(i, g)$ . Conviene cuando “ $p$  fija desconocida” y se aprende de los resultados.

**Dirichlet–Multinomial (1) Contexto.** Igual que Laplace pero con  $k$  resultados posibles (dado de  $k$  caras con sesgo desconocido,  $k$  tipos de enemigos). **(2) Modelo.**  $\theta \sim \text{Dir}(\alpha_1, \dots, \alpha_k)$ ,  $X_j \mid \theta \stackrel{\text{iid}}{\sim} \text{Cat}(\theta)$ , conteos  $c_m$  tras  $i$  observaciones,  $\alpha_0 = \sum_m \alpha_m$ . **(3) Resultado.**  $\theta \mid c \sim \text{Dir}(\alpha + c)$ ;  $\mathbb{P}(X_{i+1} = m \mid c) = \frac{\alpha_m + c_m}{\alpha_0 + i}$  (Laplace:  $\frac{c_m+1}{i+k}$ );  $\mathbb{P}(c) = \prod_m \frac{n!}{c_m!} \cdot \frac{\Gamma(\alpha_0)}{\Gamma(\alpha_0+n)} \prod_m \frac{\Gamma(\alpha_m + c_m)}{\Gamma(\alpha_m)}$ . **(4)**

**Idea.** Integral de Dirichlet  $\int_{\Delta} \prod \theta_m^{x_m-1} d\theta = \frac{\prod \Gamma(x_m)}{\Gamma(\sum x_m)}$  y conjugación; equivale a una urna de Pólya con  $\alpha_m$  bolas de cada color. **(5) CP.**  $O(k)$  por consulta o  $O(1)$  manteniendo  $i$  y  $c$ ; el estado de un DP es el vector de conteos (compresión: contar solo cuántos colores tienen cada conteo). Conviene con varias categorías y probabilidades desconocidas e intercambiables.

**Urnas de Pólya (1) Contexto.** Se saca una bola, se devuelve y se agregan  $c$  bolas del mismo color (“el rico se hace más rico”). **(2) Modelo.** Inicio con  $a$  rojas y  $b$  azules;  $R_n$  = rojas obtenidas en  $n$  extracciones. **(3) Resultado.**  $\mathbb{P}(\text{roja en } n+1 \mid R_n = r) = \frac{a+cr}{a+b+cn}$ ; una secuencia con  $r$  rojas y  $n-r$  azules tiene probabilidad  $\frac{a^{(r,c)} b^{(n-r,c)}}{(a+b)^{(n,c)}}$

con  $x^{(m,c)} = x(x+c) \cdots (x+(m-1)c)$ . Es exactamente Beta–Binomial con  $(a/c, b/c)$  y la fracción límite es Beta( $a/c, b/c$ ). **(4) Idea.** Inducción sobre la secuencia: el numerador y el denominador son productos crecientes que no dependen del orden (intercambiables). **(5) CP.**  $O(1)$  por paso, DP  $(n, r)$  en  $O(N^2)$ . Conviene cuando la probabilidad de la siguiente extracción es proporcional al conteo acumulado.

**Estadísticos de orden y espaciamentos (1) Contexto.** Se eligen  $n$  puntos al azar en  $[0, 1]$  (o una vara se parte en  $n+1$  trozos) y se pregunta por el mayor, el menor, el  $k$ -ésimo o el trozo más largo. **(2) Modelo.**  $U_1, \dots, U_n \stackrel{\text{iid}}{\sim} \mathcal{U}(0, 1)$ ,  $U_{(1)} \leq \dots \leq U_{(n)}$ , espaciamentos  $D_j = U_{(j)} - U_{(j-1)}$  con  $U_{(0)} = 0$ ,  $U_{(n+1)} = 1$ . **(3) Resultado.**  $U_{(k)} \sim \text{Beta}(k, n-k+1)$ ;  $\mathbb{E}[U_{(k)}] = \frac{k}{n+1}$ ;  $\text{Var} = \frac{k(n-k+1)}{(n+1)^2(n+2)}$ ;  $\mathbb{E}[\text{máx}] = \frac{n}{n+1}$ ,  $\mathbb{E}[\text{mín}] = \frac{1}{n+1}$ ,  $\mathbb{E}[\text{rango}] = \frac{n-1}{n+1}$ ;  $(D_1, \dots, D_{n+1}) \sim \text{Dir}(1, \dots, 1)$  (intercambia-

bles, cada una con esperanza  $\frac{1}{n+1}$ );  $\mathbb{E}[\text{máx}_j D_j] = \frac{1}{n+1} \sum_{j=1}^{n+1} \frac{1}{j}$ ;  $\mathbb{P}(U_{n+1} \text{ es el } k\text{-ésimo menor}) = \frac{1}{n+1}$ . Para otra  $F$  continua: aplicar  $F^{-1}$ . **(4) Idea.**  $\mathbb{P}(U_{(k)} \in dx) = \frac{n!}{(k-1)!(n-k)!} x^{k-1} (1-x)^{n-k} dx$  (elegir cuáles  $k-1$  caen antes de  $x$ , una cae en  $dx$ , el resto después); espaciamentos intercambiables por simetría, y  $\sum D_j = 1$  da la esperanza. **(5) CP.**  $O(1)$ . Conviene cuando aparecen máximos/mínimos de uniformes, partir intervalos al azar o “probabilidad de que el nuevo punto sea récord”.

**Suma de colas y máximo discreto (1) Contexto.** Esperanza del máximo de  $n$  dados de  $d$  caras, o de cualquier v.a. entera no negativa con cola conocida. **(2) Modelo.**  $X \geq 0$  entera (o continua). **(3) Resultado.**  $\mathbb{E}[X] = \sum_{k \geq 1} \mathbb{P}(X \geq k)$  y  $\mathbb{E}[X] = \int_0^\infty \mathbb{P}(X > t) dt$ ; para  $M = \text{máx de } n \text{ dados iid de } d \text{ caras}$ :  $\mathbb{E}[M] = \sum_{x=1}^d (1 - (\frac{x-1}{d})^n) = d - \sum_{x=1}^{d-1} (\frac{x}{d})^n$ . **(4) Idea.**  $X = \sum_{k \geq 1} \mathbf{1}[X \geq k]$  y linealidad (o Fubini en el caso continuo);  $\mathbb{P}(M \leq x) = (x/d)^n$  por independencia. **(5) CP.**  $O(d \log n)$  con exponenciación rápida. Conviene siempre que  $\mathbb{P}(X \geq k)$  es más fácil que la pmf.

**Coleccionista de cupones generalizado (1) Contexto.** Hay  $n$  tipos de cupón y cada compra da el tipo  $i$  con probabilidad  $p_i$ ; se pide el número esperado de compras hasta tener todos (o  $m$  distintos). **(2) Modelo.**  $T$  = primer instante en que todos los tipos aparecieron,  $\sum p_i = 1$ . **(3) Resultado.**  $\mathbb{E}[T] = \int_0^\infty (1 - \prod_{i=1}^n (1 - e^{-p_i t})) dt = \sum_{\emptyset \neq J \subseteq [n]} \frac{(-1)^{|J|+1}}{p(J)}$ , con  $p(J) = \sum_{i \in J} p_i$ . Caso uniforme:  $\mathbb{E} = \sum_{k=0}^{m-1} \frac{n}{n-k}$ . **(4) Idea.** Poissonización: las compras llegan como proceso de Poisson de tasa 1, el tipo  $i$  aparece como proceso independiente de tasa  $p_i$ , así que el tiempo continuo es  $\text{máx}_i \text{Exp}(p_i)$  con  $\mathbb{P}(\leq t) = \prod_i (1 - e^{-p_i t})$ ; el tiempo medio entre llegadas es 1 y por Wald  $\mathbb{E}[T_{\text{cont}}] = \mathbb{E}[T]$ . Expandir el producto da inclusión–exclusión con  $\int_0^\infty e^{-p(J)t} dt = \frac{1}{p(J)}$ . **(5) CP.** Inclusión–exclusión  $O(2^n)$  ( $n \lesssim 20$ ); si los  $p_i$  son enteros pequeños, DP sobre la suma  $p(J)$  con signo en  $O(n \cdot S)$ ; uniforme  $O(n)$ . Conviene con probabilidades no uniformes y  $n$  pequeño, o con DP por estados “cuántos faltan”.

**Identidad Min–Max (1) Contexto.** Esperanza del tiempo en que *todos* los elementos de un conjunto han ocurrido (cupones, procesos paralelos, lámparas que se apagan). **(2) Modelo.** Variables aleatorias  $X_i$  (cualquier dependencia),  $S = \{1, \dots, n\}$ . **(3) Resultado.**  $\text{máx } S = \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|+1} \text{mín } T$ , y por linealidad  $\mathbb{E}[\text{máx } S] = \sum_{T \neq \emptyset} (-1)^{|T|+1} \mathbb{E}[\text{mín } T]$ . Generalización:  $\mathbb{E}[k\text{-ésimo mayor}] = \sum_T (-1)^{|T|-k} \binom{|T|-1}{k-1} \mathbb{E}[\text{mín } T]$ . **(4) Idea.** Para reales  $x_i \geq 0$  y  $t \geq 0$ :  $\mathbf{1}[\text{máx} > t] = 1 - \prod_i (1 - \mathbf{1}[x_i > t]) = \sum_{T \neq \emptyset} (-1)^{|T|+1} \prod_{i \in T} \mathbf{1}[x_i > t] = \sum_T (-1)^{|T|+1} \mathbf{1}[\text{mín } T > t]$  (producto = indicador del mínimo), e integrar en  $t$  (o aplicar cualquier funcional lineal). **(5) CP.**  $O(2^n)$  en general; útil cuando  $\mathbb{E}[\text{mín } T]$  tiene forma cerrada (p. ej.  $\frac{1}{p(T)}$  para exponenciales o cu-

pones, con DP sobre la suma como arriba). Conviene si el mínimo de un subconjunto es fácil pero el máximo no.

**Carrera de exponenciales y Poissonización (1) Contexto.** Varios procesos independientes compiten (cola de servidores, cupones con tiempos continuos); se pide quién termina primero o cuánto tarda el último. **(2) Modelo.**  $T_i \sim \text{Exp}(\lambda_i)$  independientes,  $\Lambda = \sum \lambda_i$ . **(3) Resultado.**  $\text{mín}_i T_i \sim \text{Exp}(\Lambda)$ ;  $\mathbb{P}(T_k = \text{mín}_i T_i) = \frac{\lambda_k}{\Lambda}$ ; el mínimo es independiente de cuál lo alcanza; si  $\lambda_i = \lambda$ :  $\mathbb{E}[\text{máx}] = \frac{H_n}{\lambda}$  y  $\mathbb{E}[k\text{-ésimo en terminar}] = \frac{1}{\lambda} \sum_{j=0}^{k-1} \frac{1}{n-j}$ . En una CTMC, cada estado dura  $\text{Exp}(\Lambda_s)$  con  $\mathbb{E} = \frac{1}{\Lambda_s}$  y salta a  $s'$  con probabilidad  $\frac{\lambda_{s \rightarrow s'}}{\Lambda_s}$ . **(4) Idea.**  $\mathbb{P}(\text{mín} > t) = \prod e^{-\lambda_i t} = e^{-\Lambda t}$ ; falta de memoria  $\Rightarrow$  tras cada llegada el resto sigue siendo exponencial con los mismos parámetros. **(5) CP.**  $O(1)$  por paso del DP sobre estados; convierte problemas continuos en cadenas discretas ponderadas por  $\frac{1}{\Lambda}$ . Conviene cuando el enunciado habla de tiempos continuos o tasas.

**Ruina del jugador (1) Contexto.** Un caminante empieza en  $i$ , sube con probabilidad  $p$  y baja con  $q = 1-p$ , y se detiene al llegar a 0 o a  $N$ . **(2) Modelo.** Paseo  $S_n$  con barreras absorbentes;  $r = q/p$ ;  $P_i = \mathbb{P}(\text{llegar a } N \text{ antes que a } 0)$ ;  $T$  = tiempo de parada. **(3) Resultado.**  $P_i = \frac{1-r^i}{1-r^N}$  si  $p \neq q$  y  $P_i = \frac{i}{N}$  si  $p = q$ ;  $\mathbb{E}[T_i] = i(N-i)$  si  $p = q$ , y  $\mathbb{E}[T_i] = \frac{1}{q-p} \left( i - N \frac{1-r^i}{1-r^N} \right)$  si  $p \neq q$ . Con una sola barrera (en 0): ruina con probabilidad  $\text{mín}(1, r^i)$ . **(4) Idea.** Análisis del primer paso:  $P_i = pP_{i+1} + qP_{i-1}$ , ecuación característica  $px^2 - x + q = 0$  con raíces 1 y  $r$ ; para el tiempo  $E_i = 1 + pE_{i+1} + qE_{i-1}$  con solución particular  $\frac{i}{q-p}$ . Alternativa:  $r^{S_n}$  y  $S_n - (p-q)n$  son martingalas; teorema de parada opcional. **(5) CP.**  $O(1)$  con potencias modulares (si  $r \equiv 1 \pmod{M}$  usar el caso  $p = q$  con cuidado). Conviene en pasos aleatorios 1D con barreras; con pasos no unitarios pasa a ecuaciones lineales (siguiente ítem).

**Cadenas de Markov absorbentes (1) Contexto.** Un proceso salta entre estados con probabilidades fijas hasta caer en un estado terminal; se pide la probabilidad de terminar en cada uno o el tiempo esperado. **(2) Modelo.**  $P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$  con  $Q$  transitorio  $\rightarrow$  transitorio y  $R$  transitorio  $\rightarrow$  absorbente. **(3) Resultado.** Matriz fundamental  $N = (I - Q)^{-1} = \sum_{k \geq 0} Q^k$  ( $N_{uv}$  = visitas esperadas a  $v$  desde  $u$ ); tiempos esperados  $t = N\mathbf{1}$ ; probabilidades de absorción  $B = NR$ ; probabilidad de estar en  $s$  tras exactamente  $T$  pasos:  $e_u^T P^T$ . **(4) Idea.**  $Q^k \rightarrow 0$  (de cada estado transitorio se escapa con probabilidad positiva), luego la serie geométrica matricial converge y  $(I - Q)N = I$ ; sumar sobre columnas da el número esperado de pasos. **(5) CP.** Eliminación gaussiana  $O(s^3)$  (modular: verificar que  $I - Q$  sea invertible mod  $M$ ); matrices tridiagonales (1D) en  $O(s)$ ; horizonte  $T$  grande con exponenciación  $O(s^3 \log T)$  o Kitamasa/Berlekamp–Massey si hay recurrencia lineal; en árboles/DAG escribir  $E_v = a_v E_{\text{padre}} + b_v$  y despejar en  $O(n)$  (“incógnita lineal”). Conviene en cualquier “esperanza de pasos hasta terminar” con ciclos.

**Paseos aleatorios en grafos (1) Contexto.** Una partícula salta a un vecino uniforme; se pide tiempo esperado para llegar a  $v$ , para volver, o para visitar todo. **(2) Modelo.** Grafo conexo no dirigido con  $m$  aristas;  $h(u, v) = \mathbb{E}[\text{pasos } u \rightarrow v]$ . **(3) Resultado.** Distribución estacionaria  $\pi_v = \frac{\deg v}{2m}$ ; tiempo de retorno  $\mathbb{E}_v[T_v^+] = \frac{2m}{\deg v}$ ; tiempo de conmutación  $h(u, v) + h(v, u) = 2m R_{\text{ef}}(u, v)$ ; tiempo de cobertura  $\leq 2m(n-1)$ ; en un camino de  $n$  nodos con reflexión en 0,  $h(0 \rightarrow n) = n^2$ . **(4) Idea.**  $\pi$  satisface el balance detallado  $\pi_u / \deg u = \pi_v / \deg v$  y la esperanza del retorno es  $1/\pi_v$  (teorema de renovación); la fórmula de conmutación sale de la analogía con redes eléctricas (leyes de Kirchhoff). **(5) CP.**  $O(n^3)$  resolviendo el sistema  $h(u, v) = 1 + \frac{1}{\deg u} \sum_{w \sim u} h(w, v)$ , o  $O(n)$  en árboles. Conviene en problemas de “caminata aleatoria sobre un grafo/árbol”.

**Reflexión y teorema del escrutinio (1) Contexto.** Dos candidatos reciben  $a > b$  votos en orden aleatorio; probabilidad de que el ganador vaya siempre adelante (o nunca atrás); conteo de caminos que no tocan una barrera. **(2) Modelo.** Todas las  $\binom{a+b}{a}$  secuencias equiprobables. **(3) Resultado.**  $\mathbb{P}(\text{A siempre estrictamente adelante}) = \frac{a-b}{a+b}$ ;  $\mathbb{P}(\text{A nunca atrás}) = \frac{a-b+1}{a+1}$ ; en particular, para  $a=b=n$  la probabilidad de no pasar bajo cero es  $\frac{1}{n+1}$  (números de Catalan). **(4) Idea.** Reflexión: los caminos de  $A$  a  $B$  que tocan la recta  $y = -1$  se biyectan con los caminos de  $A'$  (reflejado) a  $B$ , y  $\binom{a+b}{a} - \binom{a+b}{a+1}$  cuenta los buenos; alternativa por rotación cíclica (lema de Dvoretzky–Motzkin). **(5) CP.**  $O(1)$  con factoriales precalculados. Conviene en caminos con restricción de signo o de barrera, y para depurar un DP  $O(n^2)$  que cuenta lo mismo.

**Linealidad de la esperanza en grafos aleatorios (1) Contexto.** Esperanza del número de componentes, aristas de un árbol aleatorio, o grados, cuando el grafo se subsampla al azar. **(2) Modelo.** Indicadores  $\mathbf{1}_e, \mathbf{1}_v$  sin necesidad de independencia entre ellos. **(3) Resultado.** Bosque con  $n$  vértices, aristas conservadas con prob.  $p$ :  $\mathbb{E}[\#\text{comp}] = n - (n-1)p$ ; vértices conservados con prob.  $p$ :  $\mathbb{E}[\#\text{comp}] = np - (n-1)p^2$  (componentes =  $V' - E'$  en un bosque).  $G(n, p)$ :  $c(n) = 1 - \sum_{k=1}^{n-1} \binom{n-1}{k-1} c(k)(1-p)^{k(n-k)}$  es  $\mathbb{P}(\text{conexo})$  y  $\mathbb{E}[\#\text{comp}] = \sum_k \binom{n}{k} c(k)(1-p)^{k(n-k)}$ . Árbol de expansión uniforme:  $\tau(G) = \det$  de cualquier cofactor del Laplaciano, y  $\mathbb{P}(e \in T) = R_{\text{ef}}(e)$ . Árbol etiquetado uniforme  $(n^{n-2}, \text{Cayley})$ :  $\deg v - 1 \sim \text{Bin}(n-2, \frac{1}{n})$ ,  $\mathbb{E}[\#\text{hojas}] = n(1 - \frac{1}{n})^{n-2} \approx \frac{n}{e}$ . **(4) Idea.** Para  $c(n)$ : fijar el componente del vértice 1 (tamaño  $k$ , conexo, sin aristas hacia el resto); linealidad  $\mathbb{E}[\sum \mathbf{1}] = \sum \mathbb{P}$ ; Kirchhoff y propiedad de Prüfer. **(5) CP.**  $O(n^2)$  para  $c(n)$ ,  $O(n^3)$  para el determinante,  $O(n)$  para bosques. Conviene cuando el enunciado pide “esperanza de cuántos...” (siempre intentar descomponer en indicadores antes de un DP).

**Linealidad en permutaciones aleatorias (1) Contexto.** Permutación uniforme de  $n$  elementos; se piden puntos fijos, ciclos, récords o inversiones esperadas. **(2) Modelo.**  $\sigma$  uniforme en  $S_n$ . **(3) Resultado.**  $\mathbb{E}[\#\text{puntos fijos}] = 1$ ;  $\mathbb{E}[\#\text{ciclos}] = H_n$ ;  $\mathbb{E}[\#\text{récords}] = H_n$  (el  $i$ -ésimo es récord con prob.  $\frac{1}{i}$ , independien-

tes);  $\mathbb{E}[\#\text{inversiones}] = \frac{n(n-1)}{4}$ ;  $\mathbb{E}[\text{longitud del ciclo de } 1] = \frac{n+1}{2}$ ;  $\mathbb{P}(\text{sin puntos fijos}) = \frac{D_n}{n!} \rightarrow \frac{1}{e}$  con  $D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$ . **(4) Idea.** Indicadores  $\mathbf{1}[\sigma(i)=i]$  con  $\mathbb{P} = \frac{1}{n}$ ;  $\mathbf{1}[\text{par}(i, j) \text{ invertido}]$  con prob.  $\frac{1}{2}$  por simetría; la longitud del ciclo de 1 es uniforme en  $\{1, \dots, n\}$ ; inclusión-exclusión para  $D_n$ . **(5) CP.**  $O(1)$  o  $O(n)$  para sumar. Conviene siempre que haya simetría entre elementos.

**Rachas, patrones y fórmula de Conway (1) Contexto.** Esperanza de lanzamientos hasta ver  $k$  éxitos seguidos, o hasta ver un patrón  $s$  (HHT, ABRACADABRA); probabilidad de que un patrón aparezca antes que otro. **(2) Modelo.** Ensayos Bernoulli( $p$ ) (o letras iid),  $T_s$  = primer tiempo en que  $s$  aparece. **(3) Resultado.** Racha de  $k$  éxitos:  $\mathbb{E}[T] = \sum_{j=1}^k p^{-j} = \frac{1-p^k}{(1-p)p^k}$ . Patrón  $s$  con letras iid:  $\mathbb{E}[T_s] = \sum_{\ell: \text{pref}_{\ell}(s)=\text{suf}_{\ell}(s)} \frac{1}{\mathbb{P}(\text{pref}_{\ell}(s))}$  (suma sobre los bordes de  $s$ , incluido  $\ell=|s|$ ); moneda justa:  $\sum 2^{\ell}$ . Penney (moneda justa): con  $L(X, Y) = \sum_k 2^{k-1} [\text{los últimos } k \text{ de } X = \text{los primeros } k \text{ de } Y]$ ,  $\mathbb{P}(A \text{ antes que } B) = \frac{L(B, B) - L(B, A)}{(L(A, A) - L(A, B)) + (L(B, B) - L(B, A))}$ . **(4) Idea.** Racha:  $E_k = E_{k-1} + 1 + (1-p)E_k$  (si falla, se reinicia). Patrón: casino con apuestas justas, un jugador entra en cada instante y apuesta todo a la siguiente letra; el capital total al detener es  $\sum_{\ell} 1/\mathbb{P}(\text{pref}_{\ell})$  para los bordes (martingala y parada opcional). **(5) CP.**  $O(k)$  o  $O(1)$  con serie geométrica; patrones: bordes con KMP/Z en  $O(|s|)$ ; “primero en aparecer” con dos patrones se resuelve con  $L$  en  $O(|A| + |B|)$ . Conviene en problemas con reinicio condicional (falla  $\Rightarrow$  volver al inicio de un autómata).

**Reinicios geométricos e identidad de Wald (1) Contexto.** Se repite un intento (que cuesta tiempo aleatorio) hasta que uno tiene éxito; se pide el costo total esperado. **(2) Modelo.** Intentos iid con costo  $C_i$  (incluye fallos) y éxito con prob.  $q > 0$ ;  $N$  = número de intentos,  $T = \sum_{i \leq N} C_i$ . **(3) Resultado.**  $\mathbb{E}[N] = \frac{1}{q}$  y  $\mathbb{E}[T] = \mathbb{E}[N]\mathbb{E}[C] = \frac{\mathbb{E}[C]}{q}$ ; en general, para un tiempo de parada  $\tau$  con  $\mathbb{E}[\tau] < \infty$ :  $\mathbb{E}[\sum_{i \leq \tau} X_i] = \mathbb{E}[\tau]\mathbb{E}[X_1]$ . **(4) Idea.**  $\sum_{i \leq \tau} X_i = \sum_{i \geq 1} X_i \mathbf{1}[\tau \geq i]$  y  $\{\tau \geq i\}$  es independiente de  $X_i$  (depende solo de  $X_1, \dots, X_{i-1}$ ); tomar esperanza. Contraejemplo: si  $\tau$  “mira hacia adelante” (p. ej.  $\tau$  = último éxito), Wald falla. **(5) CP.**  $O(1)$ . Conviene en “estrategia con reintentos” y para cerrar DP con autoloop:  $E = c + (1-q)E + \dots \Rightarrow E = \frac{c}{q}$ .

**Colisiones y paradoja del cumpleaños (1) Contexto.** Cuántos elementos caben en  $N$  casillas (hash de 64 bits, códigos, cumpleaños) antes de que haya colisión. **(2) Modelo.**  $n$  valores uniformes iid en  $\{1, \dots, N\}$ . **(3) Resultado.**  $\mathbb{P}(\text{sin colisión}) = \prod_{i=0}^{n-1} (1 - \frac{i}{N}) \leq e^{-n(n-1)/(2N)}$ ;  $\mathbb{E}[\#\text{pares colisionados}] = \binom{n}{2} \frac{1}{N}$ ;  $n \approx 1.18\sqrt{N}$  da probabilidad  $\frac{1}{2}$ ;  $\mathbb{E}[\text{tamaño hasta } 1.\text{ª colisión}] \approx \sqrt{\pi N/2}$ . **(4) Idea.** Producto de probabilidades condicionales,  $1-x \leq e^{-x}$ , y linealidad con indicadores de pares. **(5) CP.**  $O(1)$  para estimar error de hashing (p. ej.  $n = 10^6$  con  $N = 2^{64}$ : error  $\approx \frac{n^2}{2N} \approx 3 \cdot 10^{-8}$ ). Conviene al elegir módulos/bases de hashing y al analizar algoritmos aleatorizados.

**Test polinomial (Schwartz–Zippel) y verificación aleatoria (1) Contexto.** Comprobar si dos expresiones/matrices son iguales (identidad polinómica,  $AB = C$ , igualdad de multiconjuntos) sin calcularlas por completo. **(2) Modelo.**  $P \not\equiv 0$  de grado total  $d$  sobre un cuerpo  $F$ ;  $r$  uniforme en  $S^k$ ,  $S \subseteq F$ . **(3) Resultado.**  $\mathbb{P}(P(r) = 0) \leq \frac{d}{|S|}$ ; Freivalds: si  $AB \neq C$  y  $x \in \{0, 1\}^n$  uniforme,  $\mathbb{P}(ABx = Cx) \leq \frac{1}{2}$  (error  $2^{-t}$  con  $t$  repeticiones). **(4) Idea.** Inducción en el número de variables: un polinomio univariado no nulo de grado  $d$  tiene  $\leq d$  raíces; fijar las demás variables y condicionar. **(5) CP.** Freivalds  $O(n^2)$  por prueba frente a  $O(n^3)$ ; hashing polinomial de cadenas con base aleatoria. Conviene para verificar resultados caros con error unilateral controlado.

**Problema de la secretaria (parada óptima) (1) Contexto.** Se ven  $n$  candidatos en orden aleatorio y hay que aceptar o rechazar al instante; se quiere maximizar la probabilidad de elegir el mejor. **(2) Modelo.** Orden uniforme; estrategia: rechazar los primeros  $r-1$  y aceptar el primero mejor que todos los anteriores. **(3) Resultado.**  $\mathbb{P}(\text{éxito}_r) = \frac{r-1}{n} \sum_{j=r}^n \frac{1}{j-1}$  para  $r \geq 2$ ; el óptimo es  $r^* \approx \frac{n}{e}$  con probabilidad  $\rightarrow \frac{1}{e}$ . **(4) Idea.** El mejor está en la posición  $j$  con prob.  $\frac{1}{n}$ , y se acepta si el mejor de los  $j-1$  anteriores está entre los primeros  $r-1$  (prob.  $\frac{r-1}{j-1}$ ); optimizar sobre  $r$ . **(5) CP.**  $O(n)$  para probar todos los  $r$ ; para variantes (valores conocidos, costos) se usa DP hacia atrás sobre la “esperanza de seguir”. Conviene en parada óptima:  $V_i = \max(\text{aceptar}, \mathbb{E}[V_{i+1}])$ .

**Bayes con hipótesis discretas (1) Contexto.** Hay  $k$  “tipos” posibles del sistema (dado justo/cargado, jugador fuerte/débil) y se observan resultados. **(2) Modelo.** Prior  $\pi_h$ , verosimilitudes  $\ell_h(x)$ .

**(3) Resultado.**  $\mathbb{P}(h \mid x_{1:i}) = \frac{\pi_h \prod_{j \leq i} \ell_h(x_j)}{\sum_{h'} \pi_{h'} \prod_{j \leq i} \ell_{h'}(x_j)}$  y predicción  $\mathbb{P}(x_{i+1} \mid x_{1:i}) = \sum_h \mathbb{P}(h \mid x_{1:i}) \ell_h(x_{i+1})$ . **(4) Idea.** Ley de probabilidad total sobre las hipótesis y regla de Bayes; el denominador es la verosimilitud marginal. **(5) CP.**  $O(k)$  por observación; trabajar con logaritmos o renormalizar para evitar underflow, y para respuestas modulares multiplicar por inversos. Conviene cuando el enunciado dice “con probabilidad  $\pi_h$  el sistema es de tipo  $h$ ” (mezcla discreta, el análogo discreto de Laplace).

**Otros patrones que conviene tener a mano Probabilidad modular:** si la respuesta es  $P/Q$ , imprimir  $P \cdot Q^{-1}$  mód  $M$  (requiere  $Q \not\equiv 0$ ). **Esperanza condicionada:**  $\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]]$  para dividir en casos; **varianza total:**  $\text{Var } X = \mathbb{E}[\text{Var}(X \mid Y)] + \text{Var}(\mathbb{E}[X \mid Y])$ . **Funciones generadoras:**  $G_X(z) = \mathbb{E}[z^X]$  convierte sumas independientes en productos (convolución con FFT/NTT) y da  $\mathbb{E}[X] = G'_X(1)$ . **Cotas de cola:** Markov  $\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}X}{a}$ , Chebyshev  $\mathbb{P}(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$ , Chernoff para sumas de Bernoulli:  $\mathbb{P}(X \geq (1 + \delta)\mu) \leq e^{-\delta^2 \mu/3}$  ( $0 < \delta \leq 1$ ), para justificar algoritmos aleatorizados y heurísticas. **Lema de Borel–Cantelli / método probabilístico:** si  $\mathbb{E}[X] = \mu$  existe un objeto con  $X \geq \mu$  y otro con  $X \leq \mu$  (cotas de tipo “siempre existe un corte con  $\geq m/2$  aristas”). **Desaleatorización:** método de esperanzas condicionales, escoger cada decisión para no empeorar  $\mathbb{E}[X \mid \text{decisiones previas}]$  y obtener un algoritmo determinista.

## PROBABILIDAD Y PROCESOS ESTOCÁSTICOS

### ► Lemas Álgebraico-Analíticos y Series de Potencias

Para calcular valores esperados en variables aleatorias discretas no acotadas, es indispensable el cálculo exacto de series aritmético-geométricas finitas e infinitas.

- **Cálculo de la Suma**  $\sum_{i=1}^{\infty} q^i$

Para  $|q| < 1$ , la serie geométrica infinita converge a:

$$\sum_{i=0}^{\infty} q^i = \frac{1}{1-q} \implies \sum_{i=1}^{\infty} q^i = \frac{q}{1-q}$$

- **Cálculo de la Suma**  $\sum_{i=1}^{\infty} i \cdot q^i$  en General

Consideremos la función generatriz / serie de potencias  $f(q) = \sum_{i=0}^{\infty} q^i = \frac{1}{1-q}$  para  $|q| < 1$ .

Derivando término a término respecto a  $q$ :

$$\frac{d}{dq} \left( \sum_{i=0}^{\infty} q^i \right) = \sum_{i=1}^{\infty} i \cdot q^{i-1}$$

Por otro lado, la derivada de su forma cerrada es:

$$\frac{d}{dq} \left( \frac{1}{1-q} \right) = \frac{1}{(1-q)^2}$$

Igualando ambas expresiones e identificando la identidad formal:

$$\sum_{i=1}^{\infty} i \cdot q^{i-1} = \frac{1}{(1-q)^2}$$

Multiplicando toda la ecuación por  $q$ , obtenemos la fórmula general para la suma:

$$\sum_{i=1}^{\infty} i \cdot q^i = \frac{q}{(1-q)^2}$$

- **Evaluación de la Esperanza Geométrica**  $\sum_{i=1}^{\infty} i \cdot (1-p)^{i-1} p$

En procesos de tiempo de parada, se evalúa con frecuencia la serie con  $q = 1 - p$ :

$$\sum_{i=1}^{\infty} i \cdot (1-p)^{i-1} p = p \sum_{i=1}^{\infty} i \cdot (1-p)^{i-1}$$

Sustituyendo la derivada formal obtenida arriba en  $q = 1 - p$ :

$$\sum_{i=1}^{\infty} i \cdot (1-p)^{i-1} p = p \cdot \frac{1}{(1-(1-p))^2} = p \cdot \frac{1}{p^2} = \frac{1}{p}$$

### ► Distribuciones Discretas Fundamentales

- **Distribución de Bernoulli**

Modela un único ensayo aleatorio con dos posibles resultados: Éxito (1) con probabilidad  $p$ , o Fracaso (0) con probabilidad  $1 - p$ .

- **Función de Masa de Probabilidad (PMF):**  $P(X = k) = p^k (1-p)^{1-k}$ ,  $k \in \{0, 1\}$
- **Esperanza:**  $E[X] = 1 \cdot p + 0 \cdot (1-p) = p$
- **Varianza:**  $\text{Var}(X) = p(1-p)$
- **Variable Indicadora:** Si  $X_i$  es la variable indicadora de que ocurra un evento  $A$ , entonces  $E[X_i] = P(A)$ .

- **Distribución Binomial**

Modela la cantidad de éxitos  $k$  obtenidos al realizar  $n$  ensayos independientes de Bernoulli de probabilidad  $p$ .

- **PMF:**  $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$ ,  $k \in \{0, 1, \dots, n\}$
- **Esperanza:** Por linealidad de la esperanza ( $X = \sum_{j=1}^n X_j$ ):

$$E[X] = \sum_{j=1}^n E[X_j] = n \cdot p$$

- **Simetría de Transición Equiprobable** ( $p = 1/2$ ): Cuando un elemento abandona un estado hacia dos destinos equiprobables (por ejemplo, transiciones tras dejar un color  $P(\text{Verde}) = P(\text{Azul}) = 1/2$ ), la cantidad de elementos en uno de los destinos sigue una distribución Binomial( $r, 1/2$ ), cuya probabilidad para  $i$  éxitos es:

$$P(X = i) = \frac{\binom{r}{i}}{2^r}$$

- **Distribución Geométrica y Tiempo de Parada**

Modela el tiempo de parada  $T$ , definido como el número de ensayos de Bernoulli independientes con probabilidad de éxito  $p$  necesarios hasta obtener el **primer éxito**.

- **PMF:**  $P(T = i) = (1-p)^{i-1} p$ ,  $i \in \{1, 2, 3, \dots\}$
- **Esperanza (Tiempo Esperado de Parada):**

$$E[T] = \sum_{i=1}^{\infty} i \cdot P(T = i) = \sum_{i=1}^{\infty} i (1-p)^{i-1} p = \frac{1}{p}$$

- **Interpretación de Casos Favorables entre Casos Totales:** Si el espacio muestral en  $i$  pasos tiene  $M^i$  secuencias equiprobables y en los primeros  $i-1$  pasos hay  $F$  opciones de fracaso y en el  $i$ -ésimo hay  $S$  opciones de éxito (donde  $p = S/M$  y  $1-p = F/M$ ):

$$P(T = i) = \frac{F^{i-1} \cdot S}{M^i} = \left( \frac{F}{M} \right)^{i-1} \left( \frac{S}{M} \right)$$

### ► Técnicas de Análisis en Procesos Estocásticos y Cadenas de Markov

- **Análisis del Primer Paso (First-Step Analysis)**

Para encontrar el tiempo esperado de llegada  $E[u]$  desde un estado  $u$  hacia un estado terminal, se condiciona sobre la primera transición saliente:

$$E[u] = 1 + \sum_v P(u \rightarrow v) \cdot E[v]$$

Donde el +1 representa el costo/tiempo unitario consumido al realizar el paso actual.

- **Despeje de Auto-ciclos (Self-Loops)**

Si un estado  $u$  transiciona a sí mismo con probabilidad  $p_{self} = P(u \rightarrow u)$ :

$$E[u] = 1 + p_{self} \cdot E[u] + \sum_{v \neq u} P(u \rightarrow v) E[v]$$

Despejando  $E[u]$  algebraicamente para evitar recursión infinita:

$$E[u] = \frac{1 + \sum_{v \neq u} P(u \rightarrow v) E[v]}{1 - p_{self}}$$

- **Linealidad de la Esperanza y Descomposición**

Para cualquier conjunto de variables aleatorias  $X_1, X_2, \dots, X_n$  (sean independientes o dependientes):

$$E \left[ \sum_{j=1}^n X_j \right] = \sum_{j=1}^n E[X_j]$$

Permite calcular el tiempo esperado total de transformar  $N$  elementos como  $N$  veces el tiempo esperado de transformar un solo elemento, siempre que el proceso afecte a cada elemento de manera independiente.